

Syllabus — Agents IA & RAG

3 jours / 21h · Benoît Marechal · 2026

Syllabus — Agents IA & RAG

Durée : 3 jours / 21h · **Format** : 30% théorie / 70% pratique · **Présentiel ou distanciel synchrone**

Public visé & prérequis

- Développeurs, data engineers, tech leads, équipes innovation DSI
- **Prérequis** : Python intermédiaire, notions API REST, Git
- **Effectif** : 6 à 10 participants

Objectifs pédagogiques

À l'issue de la formation, le participant est capable de :

1. **Concevoir** un pipeline RAG complet (ingestion → chunking → retrieval → génération)
2. **Développer** un agent autonome utilisant des outils externes (API, DB, web)
3. **Évaluer** la fiabilité d'un système IA avec des métriques objectives (Ragas)
4. **Déployer** une démo via FastAPI / Streamlit
5. **Anticiper** coûts, latence et contraintes de conformité (RGPD, AI Act)

Modalités

Élément	Détail
Durée	21h sur 3 jours
Pédagogie	30% théorie / 70% TP individuels et collectifs
Stack	Python 3.11, Docker, VS Code, Jupyter
Modèles	OpenAI, Anthropic, Mistral, Albert (option souveraine)
Évaluation	Projet final + grille Ragas + restitution orale
Livrables	Repo Git complet + notebooks commentés + agent déployé

Jour 1 — RAG : du concept au pipeline production

Matin (3h30) · Fondamentaux & vectorisation

- Limites des LLM : fenêtre de contexte, connaissances figées, hallucinations
- **Stratégies de chunking** : fixed-size, semantic, parent-child, late chunking
- **Embeddings** : OpenAI vs Cohere vs BGE vs Mistral-embed (benchmark MTEB)
- **Bases vectorielles** : ChromaDB (local), Qdrant (souverain), Pinecone (managed) — comparatif coût/perf

TP guidé — Ingérer un corpus PDF technique avec 3 stratégies de chunking, mesurer la qualité du retrieval.

Après-midi (3h30) · Optimisation du retrieval

- **Hybrid search** : combiner BM25 + dense vectors
- **HyDE** (Hypothetical Document Embeddings) : générer une réponse fictive pour mieux chercher
- **Reranking** : Cohere Rerank, Cross-Encoders, BGE-reranker
- **Query transformation** : decomposition, multi-query, step-back prompting

TP autonome — Construire un pipeline RAG complet (10k+ chunks), benchmarker 3 stratégies de retrieval.

Livrable J1 : pipeline RAG fonctionnel + scoring de pertinence.

Jour 2 — Agents IA : autonomie et orchestration

Matin (3h30) · Frameworks & tooling

- **Comparatif** : LangChain (écosystème), LlamaIndex (RAG-first), CrewAI (multi-agents), AutoGen (Microsoft), Pydantic AI (typage strict)
- Anatomie d'un agent : **LLM + Tools + Memory + Loop**
- **Tool use** : function calling, JSON mode, Model Context Protocol (MCP)
- Patterns : ReAct, Plan-and-Execute, Reflection

Démo critique — Un "mauvais agent" qui hallucine ses outils : diagnostic et correction en live.

Après-midi (3h30) · Workflows agentiques

- **Mémoire** : court terme (conversation), long terme (vectorisée), épisodique
- **Chain of Thought** vs **Tree of Thoughts**
- Orchestration multi-agents : superviseur, hiérarchique, swarm
- Garde-fous : timeouts, max iterations, validation structurée des outputs

TP final J2 — Agent "Analyste Veille" qui cherche sur le web, croise les sources, synthétise et produit un rapport Markdown structuré.

Livrable J2 : agent autonome multi-outils avec mémoire persistante.

Jour 3 — Industrialisation & LLMOps

Matin (3h30) · Fiabilité & évaluation

- **Framework Ragas** : **faithfulness**, **answer_relevancy**, **context_precision**, **context_recall**
- **Jeux de tests** : génération automatique de questions/réponses gold
- **Observabilité** : LangSmith, Langfuse, Phoenix (Arize)
- **Coût & latence** : prompt caching, streaming, model routing, batch API
- **Conformité** : RGPD, AI Act (catégorisation des risques), souveraineté des données

TP — Évaluer le système RAG du J1 avec Ragas, identifier les régressions.

Après-midi (3h30) · Déploiement & projet final

- **API** : FastAPI (production) vs Streamlit/Gradio (démon)
- **Sécurité** : auth, rate limiting, prompt injection (OWASP LLM Top 10)
- **CI/CD** : tests automatisés sur prompts, gates qualité Ragas

Projet final (3h) — Agent “Expert Support” qui : 1. Consulte une base produit (RAG) 2. Lit l’historique client (SQL via tool) 3. Propose une réponse vérifiée et sourcée 4. Expose une API + UI Streamlit

Livrable J3 : application complète déployable + dashboard d’évaluation.

Évaluation

Critère	Pondération
Projet final fonctionnel	50%
Qualité du code & lisibilité	20%
Score Ragas (faithfulness ≥ 0.85)	20%
Restitution orale (5 min)	10%

Pour aller plus loin (optionnel)

- Mentoring 1h/personne en visio pour débloquer le projet en entreprise (J+30)
- Workshop avancé : multi-agents production-ready (1 jour)
- Audit RAG existant + plan de remédiation